

FACULTY OF ENGINEERING AND BASIC SCIENCES
ACADEMIC PROGRAM: DATA ENGINEERING AND ARTIFICIAL INTELLIGENCE
COURSE: ETL (G01)

Lab 1B – From Business Requirements to a Basic ETL Pipeline

1. Introduction

In Lab 1A, your team identified the business objectives, analytical questions, data requirements, KPIs, and expected outputs for a retail analytics platform. In this laboratory, you will build the first working version of the data system required to satisfy those needs.

The focus is not on advanced transformations or optimization. The focus is on systems thinking: designing the pipeline as a set of connected blocks, implementing each block independently, and verifying that all components work together as one end-to-end system.

In real-world Data Engineering projects, technical implementation is never the first step. Before designing databases, writing code, or building ETL pipelines, engineers must first understand the business problem they are trying to solve.

2. Learning Objectives

Implement a basic ETL pipeline that integrates heterogeneous retail data sources and produces a structured analytical repository capable of answering the business questions defined in Lab 1A.

By the end of this lab, students will be able to:

- Translate business requirements into technical pipeline components.
- Represent an ETL system using a block diagram before coding.
- Extract data from CSV, JSON, and XML sources.
- Perform basic profiling, cleaning, schema harmonization, integration, and transformation.
- Load processed data into a SQLite database.
- Execute analytical SQL queries that verify whether the business requirements were satisfied.
- Explain how each block contributes to the behavior of the complete system.

3. Dataset Description

The data cover three months (February–April 2026) and three branches. Transaction data are intentionally distributed across different formats to simulate heterogeneous operational systems.

File	Format	Purpose
sales_cali.csv	CSV	Sales transactions from Cali.
sales_bogota.json	JSON	Sales transactions from Bogotá.
sales_medellin.xml	XML	Sales transactions from Medellín.

File	Format	Purpose
products.csv	CSV	Product names, categories, list prices, and costs.
stores.csv	CSV	Store, city, and region information.
promotions.csv	CSV	Promotion dates, discount percentages, and campaigns.
monthly_targets.csv	CSV	Monthly sales target for each branch.

The transaction sources contain a small number of realistic quality issues. Students are expected to discover them through profiling rather than receiving a predefined list.

4. Dataset Description

Before implementing code, represent the solution using the following blocks:

1. Business requirements
2. Data sources
3. Extract
4. Profile
5. Clean and harmonize
6. Transform and integrate
7. Validate
8. Load
9. Query and evaluate business requirements

Your diagram must show the input and output of every block and the connection between blocks. Do not begin coding until the team can explain how the complete system should work.

5. Recommended Project Structure

```

Lab1B_ETL/
|
|— data/
|   |— raw/
|   |— processed/
|   |— output/
|
|— database/
|   |— retail_analytics.db
|
|— src/
|   |— extract.py
|   |— transform.py
|   |— load.py
|   |— queries.py
|   |— main.py

```

```

|
|— docs/
|   |— pipeline_diagram.png
|
|— README.md
|— requirements.txt
|— .gitignore

```

6. Activities

Activity 1 – Review and Trace the Requirements

Review your Lab 1A deliverable and select the six highest-priority business requirements that will be implemented in the first version of your analytical platform.

For each requirement, identify the data needed, the expected transformation, and the expected analytical output. These requirements will guide the design and implementation of your ETL pipeline.

Business Requirement	Required Data	Pipeline Block	Expected Output
Monitor total revenue and seasonal trends	POS / Transactional System (sales_cali.csv, sales_bogota.json, sales_medellin.xml)	Extract → Clean → Aggregate by date/store → Load	Aggregate table of daily and monthly sales by store and city
Identify sales performance by category	POS System + Inventory Database / Catalog (products.csv)	Extract → JOIN sales with products on product_id → Group by category	Performance dataset by category (units, revenue, rank, % of total)
Evaluate goal achievement by store and cities	POS System + ERP / Presupuestos (monthly_targets.csv)	Aggregate net_sales by store+month → JOIN with monthly_targets → Calculate compliance %	Table of Compliance Facts (actual vs target, % compliance, status)
Monitor the impact of promotions	POS System + Sistema de Promociones (promotions.csv)	JOIN sales with promotions by promotion_code and date range → Calculate ROI and sales lift	Campaign effectiveness tables (ROI, discount cost, lift vs baseline)

Activity 2 – Design the Pipeline

Create a block diagram showing all data sources, processing stages, outputs, and connections. For each block, define: purpose, input, processing responsibility, output, and possible failure.

Block	Input	Responsibility	Output	Possible Failure
Extract	Raw files: sales_cali.csv, sales_bogota.json, sales_medellin.xml	extract.py — reads each source in its native format and maps columns to the	Consolidated raw DataFrame (763 rows) with unified column names	FileNotFoundError if a source file is missing; malformed JSON/XML

	; reference CSVs (products, stores, promotions, monthly_targets)	common schema (COMMON_TRANSACTION_COLUMNS)		causing parse failure; wrong encoding
Profile	Consolidated raw DataFrame from Extract	transform.py::profile_raw_data() — counts nulls per column, duplicate IDs, invalid quantities/prices, unparseable dates, unique categoricals	Profiling summary dictionary printed to console (audit evidence)	Silent false negatives if data types are unexpected; profiling is read-only so no blocking failure
Clean/Harmonize	Raw DataFrame + profiling findings	transform.py::clean_and_harmonize_data() — strips whitespace, uppercases IDs, converts dates (mixed format), casts numeric columns, fills promotion_code nulls with "NONE", drops invalid rows	Clean DataFrame (755 valid rows, down from 763)	Overly aggressive filtering drops valid records; format="mixed" may silently misparse ambiguous dates (e.g. 04-08-2026)
Transform/Integrate	Clean DataFrame + reference tables (products, stores, promotions)	transform.py::transform_and_integrate_data() — LEFT JOINS with master tables on product_id, store_id, promotion_code; calculates gross_sales, discount_amount, net_sales; extracts year/month/week/day_name	Fully integrated analytical DataFrame (755 rows, ~24 columns) with financial metrics	Unmatched foreign keys leave NaN in store_name/product_name (caught by assertions); incorrect discount_rate cascades into wrong net_sales
Validate	Integrated DataFrame	Assert statements inside transform_and_integrate_data(): uniqueness of sale_line_id, no nulls in net_sales, no negative net_sales, no unmapped store_name or product_name	Pipeline halts with descriptive AssertionError if any invariant is violated	Assertions raise uncaught exceptions that stop the pipeline — intentional fail-fast behavior
Load	Validated integrated DataFrame + reference DataFrames	load.py — saves integrated_sales.csv to data/processed/; writes sales_analytics table to SQLite; writes all	retail_analytics.db with tables: sales_analytics, products, stores, promotions, monthly_targets	Disk full or permission error; if_exists="replace" means a partial run deletes the previous

		reference tables to SQLite		good data before the new write completes
Query	retail_analytics.db (SQLite)	queries.py — executes 7 SQL queries covering REQ 1–4; saves each result as a named table (REQ1_daily_sales_aggregate, REQ2_category_performance, REQ3_compliance_facts, REQ4_campaign_effectiveness, etc.)	7 analytical tables in SQLite + printed console report verifying all 4 business requirements	Missing table if a prior step failed; window functions (RANK, SUM OVER) require SQLite ≥ 3.25 — older versions raise OperationalError

Activity 3 – Extract

Implement one extraction function for each transaction format and functions for the reference tables. Every transaction source must be converted to a pandas DataFrame with a common schema.

Common Field	Meaning
sale_line_id	Unique sale-line identifier
sale_date	Transaction date
store_id	Store identifier
product_id	Product identifier
quantity	Units sold
unit_price	Price before discount
promotion_code	Applied promotion, when applicable
payment_method	Payment method

The extraction block must not perform business calculations. Its responsibility is to read each source and expose the data in a common technical structure. No cleaning, validation, or business transformations should be performed during the extraction stage.

Activity 4 – Profile the Extracted Data

Before cleaning or transforming, generate a compact profiling summary for the combined transaction data: row count, columns and data types, missing values, duplicate sale_line_id values, invalid quantities, invalid prices, invalid dates, and distinct values for selected categorical fields.

Record the findings in the README and explain which cleaning decisions are derived from each finding.

Activity 5 – Clean and Harmonize

Cleaning improves data quality. Apply only the rules supported by the profiling findings:

- Standardize column names and ID values.
- Trim whitespace and standardize text casing.
- Parse the different date formats into one date type.
- Convert quantity and unit_price to numeric values.
- Remove duplicated sale_line_id records, keeping the first valid occurrence.
- Reject records with invalid dates, quantity ≤ 0 , or unit_price ≤ 0 .
- Represent missing promotion codes consistently.

All cleaning decisions must be justified using the profiling results obtained in Activity 4.

Activity 6 – Transform and Integrate

Transformation changes data so that it can satisfy analytical requirements. Integrate the transactions with the reference tables and create:

- product_name and category from products.csv
- store_name, city, and region from stores.csv
- discount_pct and campaign_name from promotions.csv
- gross_sales = quantity $\times$ unit_price

- $\text{discount_amount} = \text{gross_sales} \times \text{discount_pct}$
- $\text{net_sales} = \text{gross_sales} - \text{discount_amount}$
- month, week, and day_name from sale_date
- sales_target from monthly_targets.csv

If you considered, you could add other columns to the integrated sales table.

Only transformations required to satisfy the selected business requirements should be implemented.

Activity 7 – Validate

Before loading, verify at minimum:

- sale_line_id is unique.
- Required identifiers and dates are not null.
- quantity, unit_price, gross_sales, and net_sales are positive.
- Every product matches the product master.
- Every store matches the store master.
- net_sales equals gross_sales minus discount_amount.

If a critical validation fails, the pipeline should stop before loading and record the reason in the log. Validation is the final quality gate before loading the analytical repository.

Activity 8 – Load

Save the processed dataset as data/processed/integrated_sales.csv and load it into a SQLite database named retail_analytics.db using a table named sales_analytics. The database is the analytical output of the pipeline; the final queries must read from SQLite, not from the raw files.

Activity 9 – Analytical Queries

Implement a simple menu or script that shows selected business requirements selected.

Each query must include a short explanation connecting the result to a business question from Lab 1A.

Activity 10 – Orchestrate the Complete System

Implement main.py as the entry point. It must call the blocks in the correct order, record start/end messages and row counts, stop on critical failures, and produce the CSV, SQLite database, and log file. Run the complete pipeline from the project root using one command.

Activity 11 – Verification of Requirements

Complete the table after executing the pipeline.

Business Requirement	Evidence Produced	Satisfied?	Explanation
Monitor total revenue and seasonal trends	Query showing total net sales from the sales_analytics table and the generated integrated_sales.csv.	Yes	The ETL pipeline calculates gross_sales, discount_amount, and net_sales, allowing managers to monitor total revenue.

Business Requirement	Evidence Produced	Satisfied?	Explanation
Identify sales performance by category	SQL query grouped by category and integrated dataset containing product categories.	Yes	The pipeline joins transaction data with products.csv, making it possible to analyze sales by category and identify the best and worst performing categories.
Evaluate goal achievement by store and cities	SQL query comparing net_sales with sales_target grouped by store and city.	Yes	After integrating monthly_targets.csv and stores.csv, the pipeline allows performance evaluation against business targets.
Monitor the impact of promotions	SQL query comparing sales with discount_pct and campaign_name.	Yes	Promotion data are integrated into the final dataset, allowing campaign effectiveness to be analyzed.

Activity 12 – Reflection Questions

1. How did the requirements from Lab 1A influence the design of the pipeline?

R/ The business requirements defined in Lab 1A determined every stage of the ETL pipeline. They identified the necessary data sources, the transformations required, the validation rules, and the analytical outputs. Instead of building a generic ETL process, the pipeline was specifically designed to answer the business questions and generate the KPIs requested by the client.

2. What is the difference between profiling, cleaning, transformation, and validation in your implementation?

R/ **Profiling** analyzes the extracted data to detect missing values, duplicates, invalid formats, and data quality issues.

Cleaning corrects those issues by removing duplicates, converting data types, standardizing formats, and eliminating invalid records.

Transformation enriches the data by joining transaction records with reference tables and calculating business metrics such as gross sales, discount amount, and net sales.

Validation verifies that the processed data satisfy business and quality rules before loading them into the analytical database.

3. Why was it necessary to design the system as blocks before coding?

R/ Designing the pipeline as independent blocks made the system easier to understand, implement, test, and maintain. Each block has a clear responsibility, allowing errors to be isolated more easily and enabling modifications without affecting the entire pipeline. This modular approach

also improves scalability and code organization.

4. Which block would be most affected if a branch changed its file format?

R/ The **Extract** block would be the most affected because it is responsible for reading data from the original files. If one branch changes from CSV to another format such as JSON or XML, only the extraction logic needs to be updated, while the remaining blocks can continue processing the standardized DataFrame.

5. Did the team build an ETL pipeline, or did it build a system to solve a business problem? Explain.

R/ The team built an **ETL pipeline as part of a complete business solution**. The ETL process extracts, cleans, transforms, validates, and loads the data, but its real purpose is to solve the company's business problem by providing reliable information for decision-making. The final analytical repository and SQL queries allow managers to monitor sales, evaluate store performance, analyze promotions, and support strategic decisions, which were the objectives identified in Lab 1A.